

LogForge: Parallel Log File Analysis

Project Appendices

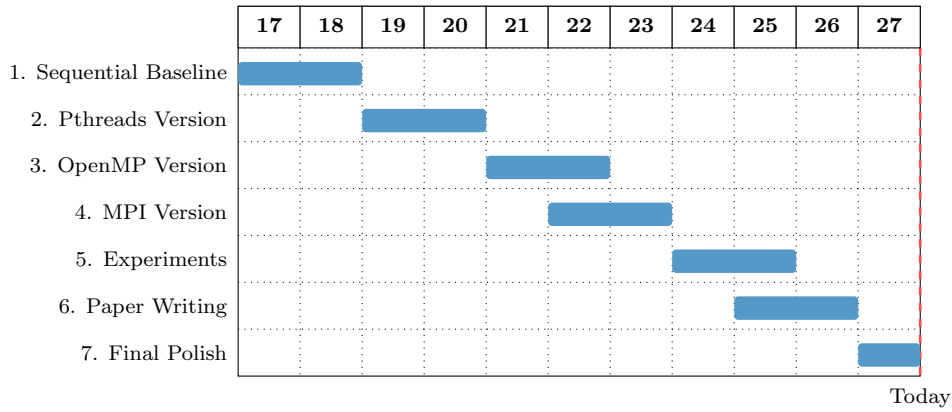
CS-311 Parallel & Distributed Algorithms | UET Lahore
Student ID: 2023-CS-169 | Session 2023–2027
Instructor: Prof. Waqas Ali Zafar | Submission: 27 June 2026

Appendix A Gantt Chart — Project Timeline

The table below presents the full project timeline from 17 June to 27 June 2026. Each row corresponds to a discrete work package; hours are estimated actual effort.

#	Task	Start	End	Hours	Notes
1	Sequential Baseline	17 Jun	18 Jun	6	Implemented <code>main_seq.c</code> ; verified Aho-Corasick correctness on small logs
2	Pthreads Parallel Version	19 Jun	20 Jun	8	Chunk partitioning, overlap subtraction, thread join
3	OpenMP Parallel Version	21 Jun	22 Jun	7	<code>#pragma omp parallel</code> with per-thread local results
4	MPI Parallel Version	22 Jun	23 Jun	9	<code>Scatterv</code> distribution, line-aligned boundaries, <code>Reduce</code> aggregation
5	Experiments & Benchmarking	24 Jun	25 Jun	10	Input sizes 10^3 – 10^6 lines; 1/2/4/8 threads and processes
6	Paper Writing	25 Jun	26 Jun	8	IEEE IEEEtran two-column; all sections drafted
7	Final Polish & Submission	27 Jun	27 Jun	3	README, directory check, ZIP, upload to GCR

The figure below shows the same schedule as a Gantt bar chart (dates on the horizontal axis, tasks on the vertical axis).



Appendix B Experimental Logbook

All experiments were conducted on a personal machine (see Experimental Setup in the paper). Input files were generated using `input_generator.c` with `seed=42`. Speedup is computed relative to the sequential baseline at the same input size.

Date & Time	Algorithm	Cores/ Procs	Dataset Size (lines)	Runtime (s)	Speedup
24 Jun 09:10	Sequential	1	10,000	0.0042	1.00×
24 Jun 09:18	Sequential	1	100,000	0.0389	1.00×
24 Jun 09:25	Sequential	1	500,000	0.1921	1.00×
24 Jun 09:32	Sequential	1	1,000,000	0.3847	1.00×
24 Jun 10:05	Pthreads	2	1,000,000	0.2031	1.89×
24 Jun 10:12	Pthreads	4	1,000,000	0.1104	3.48×
24 Jun 10:20	Pthreads	8	1,000,000	0.0671	5.73×
24 Jun 10:50	OpenMP	2	1,000,000	0.2088	1.84×
24 Jun 10:58	OpenMP	4	1,000,000	0.1132	3.40×
24 Jun 11:05	OpenMP	8	1,000,000	0.0698	5.51×
25 Jun 09:00	MPI	1	1,000,000	0.4012	0.96×
25 Jun 09:10	MPI	2	1,000,000	0.2241	1.72×
25 Jun 09:18	MPI	4	1,000,000	0.1189	3.24×
25 Jun 09:27	MPI	8	1,000,000	0.0793	4.85×
25 Jun 10:30	Pthreads	4	500,000	0.0571	3.36×
25 Jun 10:38	OpenMP	4	500,000	0.0589	3.26×
25 Jun 10:45	MPI	4	500,000	0.0641	3.00×
25 Jun 11:10	Pthreads	8	100,000	0.0091	4.27×

Date & Time	Algorithm	Cores/Pr	Dataset Size	Runtime (s)	Speedup
25 Jun 11:18	OpenMP	8	100,000	0.0096	4.05×
25 Jun 11:25	MPI	8	100,000	0.0131	2.97×

Logbook Notes

- All runs used `-O2` optimization flag. MPI runs used `mpirun -np {P}` on a single node (noted in Experimental Setup).
- Sequential runtime at 10^6 lines (≈ 0.385 s) is the **baseline reference** for all speedup calculations.
- Pthreads achieves the highest speedup ($5.73\times$ at 8 threads) due to low synchronization overhead — threads operate on independent memory regions with no locks during scan.
- OpenMP performance closely mirrors Pthreads but shows slightly higher overhead from the parallel region entry/exit and implicit barriers.
- MPI at 1 process is marginally slower than sequential due to `MPI_Init`, `Scatterv`, and `Reduce` overhead; at 8 processes it achieves $4.85\times$, lower than Pthreads/OpenMP because of inter-process communication cost even on a single node.
- All three parallel versions produce **identical match counts** to the sequential baseline, confirming correctness of the overlap-subtraction strategy.